

S4U

STEM4U — LEGO Computer Science & AI · Grades 3-5 · 30-Lesson Plan

Course Overview

Thirty weekly 60-minute sessions built on the LEGO® Education Computer Science & AI Kit (45521), coded with blocks on LEGO's Coding Canvas. Small groups of 4–6 students work in pairs (Driver/Navigator), single grade band. Real tools. Real teams. Real engineers.

By the end, your child can independently build and code a machine that senses its surroundings, decides what to do, counts and remembers, and responds to an AI model they trained themselves.

AI runs through every unit. Kids don't wait until Lesson 26 to meet artificial intelligence. Every single lesson includes a 2-minute "AI Connection" that ties the day's coding idea to how real AI works — so by the time they train their own classifier in the final unit, it feels like a payoff they've been building toward all year, not a surprise.

Scope & sequence

Unit	Lessons	Big idea	What your child can do by unit's end
1 · Basics	1–5	A program is an ordered list of steps	Plan, build, and code a machine that runs a sequence they designed
2 · Events	6–10	Programs can start from a trigger	Make a model react to buttons, taps, and sensors
3 · Loops	11–15	Computers repeat work for us	Build a machine that repeats patterns using loops
4 · Conditionals	16–20	Programs make decisions	Code a model that decides between actions on its own
5 · Variables	21–25	Programs store and use data	Build an interactive game that keeps score and reacts
6 · AI & Data	26–30	Machines can learn from examples	Train a custom AI classifier and use it to drive a build

Standards & bridges: Lessons are aligned to the CSTA K-12 CS Standards — primarily Algorithms & Programming (1B-AP), with connections to Data & Analysis (1B-DA) and Impacts of Computing (1B-IC). (See the **Standards Crosswalk** at the end for the specific codes per unit.) The lessons deliberately build the mental models — sequencing, events, loops, conditionals, variables, data — that bridge toward Scratch and, later, Python.

Rotate the "Concept + demo" format so the same 10 minutes feels fresh week to week. Pick one each lesson: **(a) straight demo** (coach shows the idea), **(b) "coach makes a mistake, kids catch it"** (coach codes it wrong on purpose; pairs spot the bug), **(c) "mystery program — guess what it does before I run it"** (predict, then run), or **(d) "pair teaches pair"** (a pair who has it explains to another). Debug lessons pair naturally with (b); review lessons with (d).

Every week gets a build/ownership hit. On mostly-code weeks, spend a spare minute on a **"reconfig or make-it-yours" beat** — add or swap a brick, decorate and name the bot, or give it a personality — so no week is screen-only.

The six design challenges are themed so each feels distinct, with a varied share-out: **L5 Build-a-Bot expo** (gallery walk) · **L10 Gadget fair** ("guess my program") · **L15 Pattern-machine showcase** (gallery walk) · **L20 Maze/guard face-off** (two pairs compare solutions) · **L25 Toy-fair game day** (play each other's games) · **L30 Story film festival** (short video for the parent note).

Progress layer: Each unit ends with a design-challenge lesson (5, 10, 15, 20, 25, 30). At each of these, the coach sends home a **short parent note** ("here's what your child mastered this unit") plus a **photo of the design-challenge build**, so families can see growth across the year.

How to read a lesson (for coaches with no CS background)

Every lesson follows the same shape and the same clock, so you can teach it cold: an "I can..." goal, a 2-line AI Connection to read aloud, a materials note (most weeks the model is already built from earlier in the unit), a **timed 60-minute flow**, Driver/Navigator pairing, differentiation (Support / Extension / Reteach), an exit check, and an optional 10-minute at-home idea. Extensions double as **fast-finisher** tasks; the Reteach line is your quick fix if a pair is stuck.

The 60-minute clock (every lesson): Warm-up **8** · Concept + demo **10** · Build/Code **30** · Challenge **7** · Share + pack-up **5**.

Model reuse: Each unit's LEGO model is built once, in the unit's first lesson, and stays assembled for the rest of the unit. Later lessons in a unit are mostly coding (with only small reconfigurations), so class time goes to thinking, not rebuilding.

Pairing: Driver runs the device/blocks; Navigator reads instructions and checks the plan. They **swap at the halfway mark** of Build/Code every lesson. Odd number of kids? Make one **rotating trio** — three roles (Driver, Navigator, Tester) that rotate at each third.

Lesson 1 — Basics: Logistics & First Run

Objective (I can...): I can pair the hub over Bluetooth, find my way around the Coding Canvas, and run a 2-block program. **CS concept:** What a program is (an ordered list of instructions a machine follows) · **Builds on:** No prior lesson — course launch.

AI Connection (2 min): Your phone unlocking with your face is a program plus AI making a choice. *First-principles question:* "If a computer had to 'recognize a cat,' what small steps might it break that into?" (Just get ideas — we'll build toward answering it all year.)

Materials: LEGO CS & AI kit (1 per pair), a device per pair (Coding Canvas), a printed "kit parts" picture card per pair. **No full build today** — use the kit's minimal/pre-built starter model (hub + one motor already attached, or the quick-start model). The unit's real model is built next lesson.

60-minute flow

Warm-up (8): "Robot chef" — a student gives the coach spoken steps to make a pretend sandwich; the coach follows them literally, mistakes and all. Point: machines do exactly what you say.

Concept + demo (10): Turn on the hub and show pairing it over Bluetooth. Tour the Canvas: block palette, workspace, Run and Stop. Drag a start block + one motor block; press Run so a real motor moves.

Build/Code (30): Each pair (1) pairs their hub over Bluetooth, (2) takes the guided tour — find the palette, Run, Stop, (3) places 2 blocks (start + spin a motor) on the pre-built model and runs it. *Swap Driver/Navigator at 15 min.* Coach circulates to confirm every hub is paired.

Challenge (7): Add a third block so the motor stops after a moment. Change the order and predict before running.

Share + pack-up (5): Each pair runs their program and names one kit part. Cables coiled, hub off, parts in tray.

Pair roles: Driver pairs the hub and drags blocks; Navigator holds the parts card and finds each part. Swap at 15 min. (*Trio: add a Tester who presses Run and reports what happened.*)

Differentiation: *Support* — give a labeled palette card so kids can find blocks by picture. *Extension / fast-finisher* — add a sound block when the motor stops. *Reteach* — re-pair the hub together slowly; have the child name each button before pressing.

Exit check: Pair has a paired hub, ran a 2-block program that moves a motor, and can point to the hub, a motor, and the Run button. **At-home (optional):** Write 5 exact steps for brushing teeth as if for a robot. ~10 min.

Lesson 2 — Basics: Build the Model & Sequence It

Objective (I can...): I can build the unit model and put blocks in the right order so it does steps one after another. **CS concept:** Sequencing (instructions run top to bottom, in order) · **Builds on:** Lesson 1 (running a program).

AI Connection (2 min): A voice assistant answering follows an ordered recipe: hear → understand → reply. *First-principles question:* "What would happen if AI did those steps out of order?"

Materials: LEGO CS & AI kit (1 per pair), device per pair, 4 sequence picture cards per pair. **This is the unit's build lesson** — build the Basics model now and keep it assembled through Lesson 5.

60-minute flow

Warm-up (8): Card scramble — pairs put 4 out-of-order "make toast" cards in the right order and discuss how wrong order breaks the result.

Concept + demo (10): Show two programs with the same blocks in different orders; run both so students see different outcomes. Name the top-to-bottom rule.

Build/Code (30): Pairs **build the Basics model** per the Canvas instructions (this is the one build for the unit), then write a 4-step sequence (light on → motor spins → wait → motor stops) and run it, checking each step in order. *Swap at 15 min.*

Challenge (7): Rearrange the same blocks into a new sensible behavior and describe what changed.

Share + pack-up (5): Each pair shows original vs. rearranged program. **Leave the model built** — store it on its tray.

Pair roles: During the build, Driver connects bricks while Navigator reads steps; during coding they swap. Swap at 15 min. (*Trio rotates build/read/test.*)

Differentiation: *Support* — a worked "twin" 4-step example on paper to adapt (not copy). *Extension / fast-finisher* — build a 6-step sequence. *Reteach* — number the blocks 1–4 out loud together and run one step at a time.

Exit check: Pair produces a working ordered sequence and can explain why order changed the result. **At-home (optional):** Draw a 4-step comic of waking up, in order. ~10 min.

Lesson 3 — Basics: Decomposition — Break It Down

Objective (I can...): I can break a big task into smaller steps before I code it. **CS concept:** Decomposition (splitting a problem into smaller parts) · **Builds on:** Lesson 2 (sequences).

AI Connection (2 min): To "recognize a cat," AI breaks the picture into tiny parts — edges, ears, whiskers — then combines them. *First-principles question:* "What small clues would you check to know it's a cat and not a dog?"

Materials: LEGO CS & AI kit + device per pair, sticky notes or mini whiteboard per pair. **Model reuse:** use the Basics model built in Lesson 2 (no rebuild) — today is code plus one tiny build tweak.

60-minute flow

Warm-up (8): "Program the human robot" — one kid is the robot and stands across the room; the partner gives only literal steps ("step forward, step forward, turn right") to guide them across. If a step is vague, the robot freezes. Point: machines need small, exact steps — that's decomposition.

Concept + demo (10): Coach picks a behavior (make it wave hello), says the small steps aloud, then turns each into a block. (*Rotate the format: try "mystery program — guess the steps before I run it."*)

Build/Code (30): Using the already-built model, pairs write small steps on sticky notes first, then place one block per step and run. **Make-it-yours beat:** add one small model change — an attachment or a LEGO "face" — and name the bot. *Swap at 15 min.*

Challenge (7): Add one more small step (light or sound) and note where it fits in the list.

Share + pack-up (5): Pairs show their sticky-note list next to the running (and now named) program. Model stays built.

Pair roles: Navigator writes the sticky-note steps; Driver codes them. Swap at 15 min. (*Trio: third child reads the step list back during testing.*)

Differentiation: *Support* — a half-filled step list to complete. *Extension / fast-finisher* — decompose a two-behavior task. *Reteach* — act out the behavior physically, naming each step, then map to blocks.

Exit check: Pair's written step list matches the blocks in their working program. **At-home (optional):** Break "make a bowl of cereal" into 6+ small steps. ~10 min.

Lesson 4 — Basics: Pseudocode — Plan Before You Build

Objective (I can...): I can write my plan in plain words (pseudocode) and turn it into blocks. **CS concept:** Pseudocode / algorithm planning. Writing the plan in plain words is a first taste of **abstraction — hiding the messy details** so you can think about the big steps first. · **Builds on:** Lesson 3 (decomposition).

AI Connection (2 min): Engineers write AI's behavior in plain words first ("IF you see a face, THEN wave") before real code. *First-principles question:* "Why plan in words before building?"

Materials: LEGO CS & AI kit + device per pair, a "plan sheet" per pair, a stack of blank "mission cards." **Model reuse:** same Basics model from Lesson 2 — code plus one tiny build tweak.

60-minute flow

Warm-up (8): "Secret mission cards" — each pair writes a short plain-word plan (3–4 lines) for a behavior on a mission card, then trades cards with another pair. Your card is the *other* pair's mission to build and run. Clear plain words win the mission.

Concept + demo (10): Coach writes 3–4 plain-word lines, then matches each to a block live — pseudocode is the bridge from idea to code. (*Rotate the format: try "coach makes a mistake, kids catch it" with one mismatched line.*)

Build/Code (30): Pairs build the mission card they received: write/refine pseudocode on the plan sheet, then code it line-by-line on the built model and run to check it matches. **Make-it-yours beat:** add or swap one brick so the bot fits the mission's theme. *Swap at 15 min.*

Challenge (7): Change one plan line (e.g., a longer wait) and update the matching block, keeping plan and code in sync.

Share + pack-up (5): Pairs read the mission pseudocode aloud, then run to prove the match. Model stays built.

Pair roles: Navigator owns the plan sheet; Driver translates lines to blocks. Swap at 15 min. (*Trio: third child checks each line was coded.*)

Differentiation: *Support* — sentence-framed pseudocode ("First the model . **Then it** ."). *Extension / fast-finisher* — write 6+ pseudocode lines. *Reteach* — code one plan line together, then let them do the next alone.

Exit check: Pair's pseudocode lines line up one-to-one with their working blocks. **At-home (optional):** Write plain-word pseudocode for getting ready for school (5 lines). ~10 min.

Lesson 5 — Basics: Design Challenge — Build-a-Bot Demo

Objective (I can...): I can plan, build, and code a small machine that does a sequence I chose. **CS concept:** Applying sequence, decomposition, and pseudocode together · **Builds on:** Lessons 1-4. *Also serves as flex/catch-up — if earlier lessons ran long, use today to finish and consolidate.*

AI Connection (2 min): Building an AI feature is decompose → plan → build → test, the same loop you'll use today. *First-principles question:* "Which of those steps do you think AI engineers redo the most?"

Materials: LEGO CS & AI kit + device per pair, plan sheets, a simple challenge card. **Model already built** (Basics model from Lesson 2) — so the 30 minutes is plan + code + test, not build-from-scratch. Pairs may make small reconfigs only.

60-minute flow

Warm-up (8): Review carousel — fast prompts: What is a sequence? Decomposition? Pseudocode?

Concept + demo (10): Present the open challenge: make the model do 4+ ordered actions of your choice. Coach models pseudocode for one idea, then hands off.

Build/Code (30): Pairs decompose, write pseudocode, and code a 4+ step sequence on the already-built model (small reconfigs allowed). Coach coaches, doesn't solve. *Swap at 15 min.*

Challenge (7): Add one "surprise" action (light/sound/extra move) and update the plan to match.

Share + pack-up (5): Build-a-Bot expo — a gallery walk: bots on trays, pairs rotate to see each other's and each pair names the concept that helped most. **Coach photographs each build for the unit parent note.**

Pair roles: Co-plan, then Driver codes / Navigator checks plan; swap at 15 min. *(Trio rotates plan/code/test.)*

Differentiation: *Support* — a 3-action target plus a partial plan to finish. *Extension / fast-finisher* — require 6 actions with a light and a sound. *Reteach* — hand them their Lesson 4 pseudocode as a starting template.

Exit check: Pair delivers a working 4+ step program with matching pseudocode and can explain their plan.

Progress layer: Send home the **Unit 1 parent note** ("your child can plan and code a sequence") with the **build photo**. **At-home (optional):** Sketch a dream robot and label 4 things it does in order. ~10 min.

Lesson 6 — Events: Build the Model & First Trigger

Objective (I can...): I can build the events model and make my program start when something happens. **CS**

concept: Events (a trigger that starts code) · **Builds on:** Lesson 5 (sequences run from a start).

AI Connection (2 min): AI reacts to what it senses — a camera noticing a face is an event. *First-principles question:* "What could our robot 'notice' that would count as an event?"

Materials: LEGO CS & AI kit + device per pair. **This is the unit's build lesson** — build the Events model now and keep it assembled through Lesson 10.

60-minute flow

Warm-up (8): "What made that happen?" — coach names triggers (doorbell rings → you answer); pairs match trigger to response.

Concept + demo (10): Compare "start on Run" vs. "start on an event"; demo a program that runs only when a button/tap event fires.

Build/Code (30): Pairs **build the Events model** (the unit's one build), then attach a sequence to an event block (e.g., button press) so it acts only when triggered. *Swap at 15 min.*

Challenge (7): Swap in a different trigger; predict, then test, how the start changes.

Share + pack-up (5): Pairs demo the event-triggered program and name the trigger. **Leave the model built.**

Pair roles: Driver builds/codes, Navigator reads steps and names the trigger. Swap at 15 min. (*Trio rotates.*)

Differentiation: *Support* — a labeled palette card showing the event blocks. *Extension / fast-finisher* — use two different events. *Reteach* — physically press the button and watch nothing happen without the event block, then add it together.

Exit check: Pair's program runs only when the chosen event fires, not on its own. **At-home (optional):** List 5 events at home and what each triggers. ~10 min.

Lesson 7 — Events: Button & Tap Triggers

Objective (I can...): I can make my model respond to a button press or a screen tap. **CS concept:** Input events from a control · **Builds on:** Lesson 6 (events as start signals).

AI Connection (2 min): Tapping a smart speaker or saying "Hey" is an input event that wakes the AI. *First-principles question:* "Why might a device wait for a signal instead of always listening?"

Materials: LEGO CS & AI kit + device per pair. **Model reuse:** Events model from Lesson 6 — code only.

60-minute flow

Warm-up (8): Human buttons — one student is a "button," another the "machine" that acts only when pressed. Reinforce input → action.

Concept + demo (10): Demo two event blocks (physical press and screen tap) driving two different actions.

Build/Code (30): On the built model, pairs program two triggers — one press event, one tap event — each starting a different short action. *Swap at 15 min.*

Challenge (7): Make one press do a two-step action (move then light) while a tap does something else.

Share + pack-up (5): Pairs demo both triggers and explain which input does what. Model stays built.

Pair roles: Driver codes trigger 1, Navigator plans trigger 2; swap at 15 min so each builds one. (*Trio: third owns testing both.*)

Differentiation: *Support* — one trigger only, with a twin example to adapt. *Extension / fast-finisher* — add a third distinct trigger. *Reteach* — do one trigger end-to-end together, then let them mirror it for the second.

Exit check: Pair shows two different inputs producing two different actions reliably. **At-home (optional):** Design a paper remote; label what each button triggers. ~10 min.

Lesson 8 — Events: Sensor Triggers (Distance & Color)

Objective (I can...): I can make my model react when a sensor detects something. **CS concept:** Sensor-driven events · **Builds on:** Lesson 7 (button/tap inputs).

AI Connection (2 min): A self-driving car's camera "seeing" a stop sign is a sensor event, just like your color sensor spotting a card. *Impacts prompt:* "Who does a self-driving car help? Could it bother or worry anyone?"

Materials: LEGO CS & AI kit + device per pair, a small object or color card to trigger the sensor. **Model reuse:** Events model from Lesson 6; add/attach the sensor as a small reconfig if needed — no full rebuild.

60-minute flow

Warm-up (8): "Sensors around us" — pairs name real sensors (automatic doors, hand dryers) and what each detects.

Concept + demo (10): Show a distance or color sensor block firing an event when an object nears or a color appears.

Build/Code (30): Pairs program the model so an action happens when the sensor detects an object/color (hand near → light on). *Swap at 15 min.*

Challenge (7): Change the trigger distance or color; test how sensitivity changes behavior.

Share + pack-up (5): Pairs demonstrate the sensor triggering their action. Model stays built.

Pair roles: Driver codes, Navigator operates the trigger object and observes. Swap at 15 min. (*Trio rotates code/trigger/observe.*)

Differentiation: *Support* — sentence-framed pseudocode ("WHEN the sensor sees , **THEN** ") to fill in.

Extension / fast-finisher — two sensor triggers doing different things. *Reteach* — watch the sensor's live reading on screen while moving the object closer.

Exit check: Pair's model reliably reacts to the sensor event, not to Run. **At-home (optional):** Spot 3 sensors at home; write what each detects. ~10 min.

Lesson 9 — Events: Multiple Events at Once

Objective (I can...): I can set up more than one event so my model can respond in different ways. **CS concept:** Parallel/multiple event handlers · **Builds on:** Lesson 8 (sensor events).

AI Connection (2 min): A smart home watches many things at once — the door, the temperature, your voice.

First-principles question: "How can one system pay attention to several events at the same time?"

Materials: LEGO CS & AI kit + device per pair, trigger objects/color cards. **Model reuse:** Events model from Lesson 6 — code only.

60-minute flow

Warm-up (8): "Many buttons" — coach calls out mixed triggers rapidly; pairs shout the matching response.

Concept + demo (10): Build two separate event stacks (a tap and a sensor) in one program; show both work independently.

Build/Code (30): Pairs program at least two independent event stacks (tap → spin; object near → sound) on the built model. *Swap at 15 min.*

Challenge (7): Add a third event; test triggering them in different orders.

Share + pack-up (5): Pairs demo each event firing on its own. Model stays built.

Pair roles: Each partner owns one event stack; swap at 15 min to review each other's. (*Trio: three stacks, one each.*)

Differentiation: *Support* — a worked twin event stack to copy the pattern from. *Extension / fast-finisher* — trigger two events nearly at once and observe. *Reteach* — build one stack, test it, then duplicate and change it for the second.

Exit check: Pair's program contains two+ event stacks that each work when triggered. **At-home (optional):** Draw a machine that responds to 3 different events. ~10 min.

Lesson 10 — Events: Design Challenge — Interactive Alarm

Objective (I can...): I can design a model that responds to at least two events I choose. **CS concept:** Applying events to a purposeful build · **Builds on:** Lessons 6–9. *Also serves as flex/catch-up if the unit ran long.*

AI Connection (2 min): A security camera with AI reacts to two events — motion AND a recognized face. *First-principles question:* "Should both events be needed, or just one, to sound your alarm?"

Materials: LEGO CS & AI kit + device per pair, plan sheets, trigger objects. **Model already built** (Events model from Lesson 6) — the 30 min is plan + code + test, small reconfigs only.

60-minute flow

Warm-up (8): Rapid review — pairs list every event type used (press, tap, distance, color).

Concept + demo (10): Challenge: build an "alarm" or "greeter" that reacts to two triggers. Coach models a pseudocode plan for one trigger, then steps back.

Build/Code (30): Pairs plan and code two event-driven responses on the built model (motion near → alarm sound; tap → reset light). *Swap at 15 min.*

Challenge (7): Add a third trigger or make one response multi-step.

Share + pack-up (5): Gadget fair — "guess my program" — before a pair triggers their gadget, the audience guesses which events it responds to, then the pair triggers them live to check. **Photograph each build.**

Pair roles: Co-plan; Driver codes / Navigator checks plan; swap at 15 min. (*Trio rotates plan/code/test.*)

Differentiation: *Support* — a one-event target with a partial plan to finish. *Extension / fast-finisher* — require distinct sensor + control triggers. *Reteach* — reuse a Lesson 7–9 program as a starting template.

Exit check: Pair delivers a working model with two+ distinct event responses and a matching plan. **Progress layer:** Send home the **Unit 2 parent note** ("your child can make a model sense and react") with the **build photo**. **At-home (optional):** Invent an alarm for something at home; list its triggers. ~10 min.

Lesson 11 — Loops: Build the Model & Repeat It

Objective (I can...): I can build the loops model and use a repeat loop so I don't copy the same block many times. **CS concept:** Loops / repetition (repeat N times). A loop is **abstraction — hiding the messy details**: one "repeat 5x" block stands in for five copied blocks. · **Builds on:** Lesson 10 (events start actions we now repeat).

AI Connection (2 min): AI gets better by repeating — training means showing examples over and over. *First-principles question:* "Why would more tries help a computer learn?"

Materials: LEGO CS & AI kit + device per pair. **This is the unit's build lesson** — build the Loops model now and keep it assembled through Lesson 15.

60-minute flow

Warm-up (8): "Do it again" — clap 10 times, then find a shorter instruction. Reveal: "repeat 10 times" is a loop.

Concept + demo (10): Show 4 copied move blocks, then replace them with one move inside a repeat-4 loop; run both to show they match.

Build/Code (30): Pairs **build the Loops model** (the unit's one build), then program a repeated action with a repeat loop (spin, pause, repeat 5x). *Swap at 15 min.*

Challenge (7): Change the repeat number; predict how the behavior length changes before running.

Share + pack-up (5): Pairs show their loop and state the count. **Leave the model built.**

Pair roles: Driver builds/codes, Navigator counts the repeats aloud. Swap at 15 min. (*Trio rotates.*)

Differentiation: *Support* — a labeled palette card highlighting the repeat block. *Extension / fast-finisher* — put two blocks inside the loop. *Reteach* — count the repeats out loud together on the first run.

Exit check: Pair replaces repeated blocks with a working repeat loop and can state the count. **At-home (optional):** Find something you do "over and over"; write it as "repeat ___ times." ~10 min.

Lesson 12 — Loops: Loops with Motion & Sound (+ Debug)

Objective (I can...): I can put several actions inside one loop so a pattern repeats. **CS concept:** Multi-action loop body · **Builds on:** Lesson 11 (single-action repeat).

AI Connection (2 min): Each training "round" repeats the same steps on new examples until the AI improves. *First-principles question:* "What might get better each time the loop runs again?"

Materials: LEGO CS & AI kit + device per pair. **Model reuse:** Loops model from Lesson 11 — code only.

60-minute flow

Warm-up (8): Pattern clap — coach does "clap-stomp-clap-stomp"; pairs find the repeating chunk that could go in a loop.

Concept + demo (10): Build a loop body with move + sound + wait; run it and point out the whole chunk repeats.

Build/Code (30): Pairs code a 3-action repeating pattern inside one loop (spin → beep → wait). **Debug mini-task (3 min):** hand each pair a "broken" program where a block sits *outside* the loop — they find and fix it so the whole pattern repeats. *Swap at 15 min.*

Challenge (7): Add a light or reverse move to the loop body; re-run.

Share + pack-up (5): Pairs demo the pattern and name the actions inside the loop. Model stays built.

Pair roles: Driver codes the loop body, Navigator lists the actions to include. Swap at 15 min. (*Trio: third leads the debug hunt.*)

Differentiation: *Support* — a 2-action loop body with a twin example. *Extension / fast-finisher* — a 4-action loop body. *Reteach* — build the body outside a loop first, confirm it works, then drag it inside.

Exit check: Pair runs a loop whose body has 3+ actions that repeat together. **At-home (optional):** Make up a 3-move dance; repeat it 4 times. ~10 min.

Lesson 13 — Loops: Forever & Stop

Objective (I can...): I can use a forever loop and know how to stop it. **CS concept:** Infinite loops and controlled stopping · **Builds on:** Lesson 12 (loop bodies).

AI Connection (2 min): An always-on assistant runs a forever loop, listening until you speak. *Impacts prompt:* "Who does an always-listening speaker help? Could it bother anyone in the house?"

Materials: LEGO CS & AI kit + device per pair. **Model reuse:** Loops model from Lesson 11 — code only.

60-minute flow

Warm-up (8): "Never stop" game — start a chant with no end; ask how to stop it. Introduce a loop that runs until told otherwise.

Concept + demo (10): Demo a forever loop (blinking light); stop it with the Stop button, then better, with an event.

Build/Code (30): Pairs program a forever loop (constant blink or spin) on the built model, then add a way to stop it (Stop button or event). *Swap at 15 min.*

Challenge (7): Use one event to start the forever loop and another to stop it.

Share + pack-up (5): Pairs demo starting and stopping. Model stays built.

Pair roles: Driver codes the loop, Navigator owns the stop plan. Swap at 15 min. (*Trio rotates start/loop/stop.*)

Differentiation: *Support* — use the Stop button only. *Extension / fast-finisher* — start and stop entirely with events. *Reteach* — run the forever loop, feel how it never ends, then add the stop together.

Exit check: Pair runs a forever loop and cleanly stops it as planned. **At-home (optional):** Name a machine that "runs forever until turned off"; how would you stop it? ~10 min.

Lesson 14 — Loops: Nested Loops (+ Debug)

Objective (I can...): I can put one loop inside another to make bigger repeating patterns. **CS concept:** Nested loops · **Builds on:** Lesson 13 (loops and stopping).

Stretch/extension, not required mastery. Nesting is a stretch goal — especially for the youngest students. A single working loop (L11–13) is the required takeaway; treat nesting as enrichment and don't hold a struggling pair to it.

AI Connection (2 min): AI training loops nest — for each round, loop through every example. *First-principles question:* "If the outer loop is 'rounds' and the inner is 'examples,' how many looks total is 3 rounds of 4 examples?"

Materials: LEGO CS & AI kit + device per pair, grid or dot paper per pair. **Model reuse:** Loops model from Lesson 11 — code only.

60-minute flow

Warm-up (8): "Rows and beats" — pairs trace a pattern that repeats within a repeat (3 rows of 4 dots) and count the total.

Concept + demo (10): Build an inner loop (beep 3×) inside an outer loop (repeat 2×); run so students hear 2 sets of 3.

Build/Code (30): Pairs code a nested loop (spin-beep 3×, all repeated 2×). **Debug mini-task (3 min):** give a program where inner and outer counts are swapped — pairs find why the total is wrong and fix it. *Swap at 15 min.*

Challenge (7): Change the inner or outer count; predict the new total before running.

Share + pack-up (5): Pairs demo and state total inner runs. Model stays built.

Pair roles: Driver builds the inner loop, Navigator adds the outer and tracks counts. Swap at 15 min. (*Trio: third leads the debug.*)

Differentiation: *Support* — a twin nested example to adapt. *Extension / fast-finisher* — add a third layer or a light in the inner loop. *Reteach* — build and test the inner loop alone first, then wrap the outer around it.

Exit check: Pair runs a working nested loop and can explain inner vs. outer counts. **At-home (optional):** Draw a 2×3 sticker pattern; write it as a loop inside a loop. ~10 min.

Lesson 15 — Loops: Design Challenge — Pattern Machine

Objective (I can...): I can design a machine that repeats a pattern I invented using loops. **CS concept:** Applying loops and nesting to a creative build · **Builds on:** Lessons 11-14. *Also serves as flex/catch-up if the unit ran long.*

AI Connection (2 min): Repetition is how AI practices — thousands of loops to get one skill right. *First-principles question:* "Would your machine improve if it 'practiced' its pattern more, like AI does?"

Materials: LEGO CS & AI kit + device per pair (with camera), plan sheets. **Model already built** (Loops model from Lesson 11) — the build time is plan + code + test, small reconfigs only.

60-minute flow *(As the flex lesson, today's clock trims Build/Code to ~20 to make room for a 10-min AI taste. If a pair needs the full build time, run the AI taste as a whole-group demo instead.)*

Warm-up (8): Loop review — pairs sort example programs into "repeat N," "forever," and "nested." *(Theme: Pattern-machine showcase — a gallery walk at the end.)*

Concept + demo (10): Challenge: build a "pattern machine" (dancer, drummer, light show) using at least one loop. Coach models pseudocode with a loop, then hands off. *(Rotate the format: try "pair teaches pair.")*

Build/Code (~20): Pairs plan and code a repeating pattern that must include a loop (bonus for nesting) on the built model. *Swap at the halfway mark.*

Meet a real AI — hands-on taste (10): Pairs open the kit's AI/classifier feature in the Coding Canvas and show cards, small objects, or hand gestures to the device camera, watching the on-screen label change. **No training yet** — just experience a classifier reacting live. *This previews Unit 6, where they'll train their own.*

Privacy note (say it aloud): the camera runs on-device, with no logins, and images are never saved.

Share + pack-up (5): Gallery walk — pairs run their pattern and point out the loop, and say one thing the classifier got right or wrong. **Photograph each build.**

Pair roles: Co-plan; Driver codes / Navigator checks the loop count; swap at 15 min. *(Trio rotates plan/code/test.)*

Differentiation: *Support* — a single-repeat target with a partial plan. *Extension / fast-finisher* — require a nested loop plus an event. *Reteach* — reuse a Lesson 12-14 loop as a template.

Exit check: Pair delivers a working pattern that uses a loop and can explain the repetition, and can say why one loop block is simpler than copying the blocks many times (**abstraction** — hiding the messy details). **Progress layer:** Send home the **Unit 3 parent note** ("your child can make a machine repeat work with loops") with the **build photo**. **At-home (optional):** Invent a light or clap pattern; write how many times each part repeats. ~10 min.

Lesson 16 — Conditionals: Build the Model & If / Then

Objective (I can...): I can build the conditionals model and use an if/then block so it decides what to do. **CS concept:** Conditionals (if a condition is true, then act) · **Builds on:** Lesson 15 (events + loops set the stage for decisions).

AI Connection (2 min): AI decides with confidence: "IF I'm 80% sure it's a dog, THEN bark." *First-principles question:* "What should happen if it's only 50% sure?"

Materials: LEGO CS & AI kit + device per pair, trigger object/color card. **This is the unit's build lesson** — build the Conditionals model now and keep it assembled through Lesson 20.

60-minute flow

Warm-up (8): "If... then..." chant — coach says conditions ("if it's raining..."); pairs finish with the action.

Concept + demo (10): Demo an if/then: if the sensor sees an object, then light on; otherwise nothing.

Build/Code (30): Pairs **build the Conditionals model** (the unit's one build), then program one if/then (object near / color seen → action). *Swap at 15 min.*

Challenge (7): Change the condition; test that the action only happens when it's true.

Share + pack-up (5): Pairs demo the true case and the "nothing happens" case. **Leave the model built.**

Pair roles: Driver builds/codes, Navigator reads the condition aloud before each run. Swap at 15 min. (*Trio rotates.*)

Differentiation: *Support* — a labeled palette card for the if/then block. *Extension / fast-finisher* — make the "then" a two-step action. *Reteach* — hold the object at/away and narrate "true... false..." while it runs.

Exit check: Pair's model acts only when the condition is true. **At-home (optional):** Write three "if... then..." rules for your house. ~10 min.

Lesson 17 — Conditionals: True or False? (+ Debug)

Objective (I can...): I can tell whether a condition is true or false and predict what my program will do. **CS concept:** Boolean conditions (true/false) · **Builds on:** Lesson 16 (if/then).

AI Connection (2 min): Under the hood, an AI's "is this a cat?" answer becomes a true/false decision after it checks its confidence. *First-principles question:* "How sure should it be before it answers 'true'?"

Materials: LEGO CS & AI kit + device per pair, true/false cards. **Model reuse:** Conditionals model from Lesson 16 — code only.

60-minute flow

Warm-up (8): True/false flash — coach reads statements ("The hub is on"); pairs flip a card.

Concept + demo (10): Show the same if/then; before each run ask "is the condition true right now?" then run to confirm.

Build/Code (30): Pairs run their if/then in a true state and a false state, predicting first each time. **Debug mini-task (3 min):** give a program whose condition is set backwards (acts when it shouldn't) — pairs diagnose and fix. *Swap at 15 min.*

Challenge (7): Add a light that shows "condition true" so the state is visible.

Share + pack-up (5): Pairs predict, then demonstrate, both a true and false outcome. Model stays built.

Pair roles: Driver runs and codes, Navigator predicts true/false before each run. Swap at 15 min. (*Trio: third leads the debug.*)

Differentiation: *Support* — a checklist for reading the condition. *Extension / fast-finisher* — use a sensor threshold as the condition. *Reteach* — watch the live sensor value and say "true/false" as the object moves.

Exit check: Pair correctly predicts and shows the action for both true and false states. **At-home (optional):** Write 5 statements about your day; mark each true or false. ~10 min.

Lesson 18 — Conditionals: If / Then / Else

Objective (I can...): I can use if/then/else so my model does one thing OR another. **CS concept:** Two-way branching (else) · **Builds on:** Lesson 17 (true/false).

AI Connection (2 min): AI branches: IF it recognizes you, THEN unlock; ELSE stay locked. *Impacts prompt:* "Face-unlock helps you get in fast — who might it help most, and could it bother or lock out anyone?"

Materials: LEGO CS & AI kit + device per pair, trigger object/color card. **Model reuse:** Conditionals model from Lesson 16 — code only.

60-minute flow

Warm-up (8): "Two doors" — conditions with two outcomes ("if green, go; else stop"); pairs act out both branches.

Concept + demo (10): Demo if/then/else: object near → green light; else → red light. Run both branches.

Build/Code (30): Pairs code an if/then/else with two clearly different outcomes from one condition. *Swap at 15 min.*

Challenge (7): Make each branch a two-step action (light + sound).

Share + pack-up (5): Pairs demo both branches and say which condition chose which. Model stays built.

Pair roles: Driver codes the "then," Navigator plans the "else"; swap at 15 min so each owns a branch. (*Trio: third owns testing both.*)

Differentiation: *Support* — the "then" filled in; students add only the "else," framed as pseudocode. *Extension / fast-finisher* — nest an if inside a branch. *Reteach* — trigger each branch by hand and label it before coding.

Exit check: Pair's model shows two different outcomes correctly chosen by one condition. **At-home (optional):** Write an if/then/else rule for choosing what to wear. ~10 min.

Lesson 19 — Conditionals: Sensors Make Decisions (+ Debug)

Objective (I can...): I can use a sensor value inside a condition so my model decides on its own. **CS concept:** Conditions driven by sensor thresholds · **Builds on:** Lesson 18 (if/then/else).

AI Connection (2 min): AI sets a threshold too — "IF confidence is above 70%, THEN act." *First-principles question:* "What goes wrong if the cutoff is set too low?"

Materials: LEGO CS & AI kit + device per pair, objects at different distances / color cards. **Model reuse:** Conditionals model from Lesson 16 — code only.

60-minute flow

Warm-up (8): "How close is close?" — pairs guess distances and pick a cutoff (threshold) for "near" vs. "far."

Concept + demo (10): Demo a condition using a sensor threshold: if distance < a set value, then act; else the other branch.

Build/Code (30): Pairs code a sensor-threshold if/then/else (closer than X → back away; else → wait). **Debug mini-task (3 min):** give a program whose threshold is off so it never triggers — pairs find and fix the value. *Swap at 15 min.*

Challenge (7): Tune the threshold; test how decisions change.

Share + pack-up (5): Pairs show the model deciding differently at two distances. Model stays built.

Pair roles: Driver codes, Navigator moves the object and calls the threshold. Swap at 15 min. (*Trio: third leads the debug.*)

Differentiation: *Support* — provide the threshold value. *Extension / fast-finisher* — two thresholds for three outcomes. *Reteach* — watch the live distance number and pick the cutoff together.

Exit check: Pair's model changes behavior based on the sensor crossing the threshold. **At-home (optional):** Describe a real "smart" behavior (auto lights) as an if/then/else. ~10 min.

Lesson 20 — Conditionals: Design Challenge — Smart Guard

Objective (I can...): I can design a model that makes its own decisions using conditionals. **CS concept:** Applying if/then/else with sensors · **Builds on:** Lessons 16–19. *Also serves as flex/catch-up if the unit ran long.*

AI Connection (2 min): A smart doorbell decides: IF it sees a person, THEN alert; ELSE ignore. *First-principles question:* "What sensor + rule would make YOUR guard smart?"

Materials: LEGO CS & AI kit + device per pair, plan sheets, trigger objects. **Model already built** (Conditionals model from Lesson 16) — the 30 min is plan + code + test, small reconfigs only.

60-minute flow

Warm-up (8): Conditionals review — pairs match scenarios to if/then vs. if/then/else. *(Theme: Maze/guard face-off.)*

Concept + demo (10): Challenge: build a "smart guard" or "helper" that decides between two actions using a sensor. Coach models a pseudocode plan with a condition, then hands off. *(Rotate the format: try "coach makes a mistake, kids catch it.")*

Build/Code (30): Pairs plan and code an if/then/else decision driven by a sensor on the built model. *Swap at 15 min.*

Challenge (7): Add a second condition or combine with a loop so it decides repeatedly.

Share + pack-up (5): Two ways to solve it — pair up two teams that met the *same* goal with different working programs. Each shows theirs, then together they decide which solution is simpler and say why (there's often more than one right answer). **Photograph each build.**

Pair roles: Co-plan; Driver codes / Navigator checks the condition; swap at 15 min. *(Trio rotates plan/code/test.)*

Differentiation: *Support* — a one-condition target with a partial plan. *Extension / fast-finisher* — require a loop plus if/then/else. *Reteach* — reuse a Lesson 18–19 program as a template.

Exit check: Pair delivers a model that chooses between actions correctly using a condition. **Progress layer:** Send home the **Unit 4 parent note** ("your child can make a model decide on its own") with the **build photo**. **At-home (optional):** Invent a "smart" gadget; write its main if/then/else rule. ~10 min.

Lesson 21 — Variables: Build the Model & a Box That Holds a Number

Objective (I can...): I can build the variables model, make a variable, and store a number in it. **CS concept:** Variables (a named box that holds a value) · **Builds on:** Lesson 20 (decisions we can now base on stored values).

AI Connection (2 min): AI stores what it learns as numbers/data. *First-principles question:* "What might a robot need to remember to get better at a task?"

Materials: LEGO CS & AI kit + device per pair, a labeled paper "box" per pair. **This is the unit's build lesson** — build the Variables model now and keep it assembled through Lesson 25.

60-minute flow

Warm-up (8): "Name the box" — coach holds a labeled box, drops in a number card, swaps it. A variable is a named box whose value can change.

Concept + demo (10): Create a variable "score," set it to 0, show it on screen, then change it and watch the display update.

Build/Code (30): Pairs **build the Variables model** (the unit's one build), then create a variable, set a starting number, and display it (screen or lights). *Swap at 15 min.*

Challenge (7): Set a different starting value; re-run to see the display change.

Share + pack-up (5): Pairs show their named variable and its stored value. **Leave the model built.**

Pair roles: Driver builds/codes, Navigator names the variable and reads its value. Swap at 15 min. (*Trio rotates.*)

Differentiation: *Support* — a twin example showing a named variable to adapt. *Extension / fast-finisher* — create two variables. *Reteach* — physically write a number in the paper box, then mirror it on screen.

Exit check: Pair creates a named variable and stores/displays a value in it. **At-home (optional):** Label 3 "boxes" (like "candles on cake"); give each a number. ~10 min.

Lesson 22 — Variables: Counting Up

Objective (I can...): I can change a variable's value to count events. **CS concept:** Updating a variable (increment) · **Builds on:** Lesson 21 (creating/storing a variable).

AI Connection (2 min): AI keeps a running tally — how many examples it has seen, how many it got right. *First-principles question:* "What would a robot count to know it's improving?"

Materials: LEGO CS & AI kit + device per pair, trigger object/button. **Model reuse:** Variables model from Lesson 21 — code only.

60-minute flow

Warm-up (8): Tally clap — coach adds a tally per clap; connect to "add 1 to the variable."

Concept + demo (10): Demo "change score by 1" inside an event so each press adds one; show the running total.

Build/Code (30): Pairs program a counter: each event (press or sensor) adds 1 to a variable and displays it. *Swap at 15 min.*

Challenge (7): Add a reset event that sets the counter back to 0.

Share + pack-up (5): Pairs demo counting up and resetting. Model stays built.

Pair roles: Driver codes the counter, Navigator triggers events and reads the total. Swap at 15 min. (*Trio rotates code/trigger/read.*)

Differentiation: *Support* — provide the event; students add the "change by 1." *Extension / fast-finisher* — count by 2 or track two counters. *Reteach* — press once, watch it go 0→1, then let them predict the next.

Exit check: Pair's variable increases correctly on each event and can be reset. **At-home (optional):** Count something for a minute (steps, claps); record the number. ~10 min.

Lesson 23 — Variables: Using a Variable in a Decision

Objective (I can...): I can use a variable inside an if/then so my program reacts to stored data. **CS concept:** Variables combined with conditionals · **Builds on:** Lesson 22 (counting) and Lesson 18 (if/then/else).

AI Connection (2 min): AI acts on stored data: IF the count of correct guesses is high enough, THEN it's "trained." *First-principles question:* "What number would tell a robot it's ready?"

Materials: LEGO CS & AI kit + device per pair, trigger object/button. **Model reuse:** Variables model from Lesson 21 — code only.

60-minute flow

Warm-up (8): "When does it change?" — coach counts claps with a rule ("at 5, everyone cheers").

Concept + demo (10): Demo: count presses; if score reaches a target, then celebrate (light + sound); else keep counting.

Build/Code (30): Pairs code a counter plus an if/then that triggers a special action when the variable hits a chosen value. *Swap at 15 min.*

Challenge (7): Change the target value or add an else branch for "not yet."

Share + pack-up (5): Pairs demo reaching the target and the model reacting. Model stays built.

Pair roles: Driver codes the if/then, Navigator owns the counter and counts to the target. Swap at 15 min. (*Trio rotates.*)

Differentiation: *Support* — provide the counter; students add the if/then as framed pseudocode. *Extension / fast-finisher* — two targets with different reactions. *Reteach* — set the target to 2 so it triggers fast, then raise it.

Exit check: Pair's model changes behavior when the variable reaches the target. **At-home (optional):** Write a rule like "when my reading minutes reach 20, I earn a break." ~10 min.

Lesson 24 — Variables: Score & Timer Together

Objective (I can...): I can use two variables to track more than one thing at once. **CS concept:** Multiple variables · **Builds on:** Lesson 23 (a variable in a decision).

AI Connection (2 min): AI juggles many numbers at once — accuracy, speed, how many examples per category. *First-principles question:* "Why can't one single number capture everything a robot needs to know?"

Materials: LEGO CS & AI kit + device per pair, trigger objects/buttons. **Model reuse:** Variables model from Lesson 21 — code only.

60-minute flow

Warm-up (8): "Two scoreboards" — pairs track points AND time with tallies, feeling the need for two boxes.

Concept + demo (10): Demo two variables (score and a countdown), updating each and showing both on screen.

Build/Code (30): Pairs program two variables — a score that counts up on an event and a timer/count that changes over a loop, both displayed. *Swap at 15 min.*

Challenge (7): Add an if/then that reacts when either variable hits a value (game over / win).

Share + pack-up (5): Pairs demo both variables changing and explain what each tracks. Model stays built.

Pair roles: Each partner owns one variable; swap at 15 min to review each other's. (*Trio: third owns the win/lose check.*)

Differentiation: *Support* — one variable pre-built; students add the second. *Extension / fast-finisher* — end the program on a combined condition. *Reteach* — build and test one variable fully, then copy the pattern for the second.

Exit check: Pair runs a program with two working, correctly labeled variables. **At-home (optional):** Design a simple game; list the two things it keeps track of. ~10 min.

Lesson 25 — Variables: Design Challenge — Scorekeeper Game

Objective (I can...): I can design an interactive game or gadget that stores and uses data. **CS concept:** Applying variables with events, loops, and conditionals · **Builds on:** Lessons 21–24. *Also serves as flex/catch-up if the unit ran long.*

AI Connection (2 min): A game AI tracks your score and adapts to it — data drives its choices. *Impacts prompt:* "A game that adapts to you can be fun — who does that help, and could it ever bother a player?"

Materials: LEGO CS & AI kit + device per pair, plan sheets, trigger objects. **Model already built** (Variables model from Lesson 21) — the 30 min is plan + code + test, small reconfigs only.

60-minute flow

Warm-up (8): Variables review — What is a variable? How do you change it? How can it make a decision?

Concept + demo (10): Challenge: build a game/gadget that keeps score or counts and reacts when it hits a target. Coach models a plan with one variable, then hands off.

Build/Code (30): Pairs plan and code a program using at least one variable plus an event and a conditional (a win/lose or milestone reaction) on the built model. *Swap at 15 min.*

Challenge (7): Add a second variable, a reset, or a timer.

Share + pack-up (5): Toy-fair game day — pairs rotate to actually play each other's games, then each creator explains the variable behind the scoreboard. **Photograph each build.**

Pair roles: Co-plan; Driver codes / Navigator checks the variable logic; swap at 15 min. (*Trio rotates plan/code/test.*)

Differentiation: *Support* — a counter-only target with a partial plan. *Extension / fast-finisher* — require two variables and a win condition. *Reteach* — reuse a Lesson 23–24 program as a template.

Exit check: Pair delivers a working game/gadget that stores, updates, and uses a variable in a decision.

Progress layer: Send home the **Unit 5 parent note** ("your child can build a game that keeps score and reacts") with the **build photo**. **At-home (optional):** Sketch your game's "scoreboard"; label what it tracks. ~10 min.

Lesson 26 — AI & Data: Monster Maze (pre-trained classifier)

Objective (I can...): I can use events with a pre-trained AI classifier to control a model. **CS concept:** AI classification as an event source (a trained model recognizes what the camera sees) · **Builds on:** Lesson 25 (events and decisions now driven by AI). *Privacy: AI runs on the device camera, locally, with no logins; images are never saved.*

AI Connection (2 min): This is the real thing — a classifier that already learned its categories, just like the face-recognition and cat-recognition we've talked about all year. *First-principles question:* "This model was trained by someone else; what do YOU think it took to teach it?" (Sets up training in Lesson 28.)

Materials: LEGO CS & AI kit + device per pair (Coding Canvas) with camera, the lesson's picture/object cards. **This is the AI unit's build lesson** — build the Monster model now and keep it assembled through Lesson 27.

60-minute flow

Warm-up (8): "Sort it" — pairs sort picture cards into groups (monster vs. not) and explain their rule. A classifier sorts images the same way.

Concept + demo (10): A pre-trained classifier already "knows" certain categories; demo how a recognition becomes an event. Note aloud: on-device, saves nothing.

Build/Code (30): Pairs **build the Monster Maze model** per the Canvas instructions, then program events so that when the pre-trained classifier recognizes a category, the model moves through the maze. *Swap at 15 min.*

Challenge (7): Add a second recognized category that triggers a different move or sound.

Share + pack-up (5): Pairs demo the classifier driving a maze move and name the trigger. **Leave the model built.**

Pair roles: Driver builds/codes, Navigator holds cards to the camera and confirms recognition. Swap at 15 min. (*Trio rotates build/code/test.*)

Differentiation: *Support* — one recognition-to-action link pre-built to extend. *Extension / fast-finisher* — chain two recognitions into a path. *Reteach* — watch the classifier's on-screen label change as cards are shown before wiring the event.

Exit check: Pair's model responds correctly when the pre-trained classifier recognizes the target. **At-home (optional):** List 3 ways phones or tablets "recognize" things (faces, voices). ~10 min.

Lesson 27 — AI & Data: Monster Snack Time (pre-trained classifier)

Objective (I can...): I can program actions using a pre-trained classifier's decisions. **CS concept:** Using classifier output in conditionals · **Builds on:** Lesson 26 (classifier events). *Privacy: on-device camera, no logs, images never saved.*

AI Connection (2 min): The classifier's guess feeds an IF/THEN — exactly the "IF I'm sure, THEN act" idea from the conditionals unit, now powered by AI. *First-principles question:* "Should the monster still eat if the AI is only a little bit sure?"

Materials: LEGO CS & AI kit + device per pair with camera, the lesson's food/object cards. **Model reuse:** Monster model from Lesson 26 (small reconfig into Snack Time if the build calls for it) — mostly code.

60-minute flow

Warm-up (8): "Yes food / no food" — pairs sort item cards the monster would or wouldn't eat.

Concept + demo (10): Demo the classifier recognizing an item and an if/then choosing the monster's reaction (eat vs. refuse).

Build/Code (30): Pairs program conditionals on the model: if the classifier recognizes a "snack," the monster eats (moves + sound); else it reacts differently. *Swap at 15 min.*

Challenge (7): Add a score variable that counts snacks eaten.

Share + pack-up (5): Pairs demo both reactions and explain the condition. Model stays built.

Pair roles: Driver codes the if/then, Navigator presents items and reads the label. Swap at 15 min. (*Trio: third owns the score add.*)

Differentiation: *Support* — the "if" branch provided; students add the "else." *Extension / fast-finisher* — track two categories with two reactions. *Reteach* — narrate "the AI says snack → true → eat" as they test.

Exit check: Pair's model reacts differently based on what the classifier recognizes. **At-home (optional):** Draw two things the monster would eat and two it wouldn't, and why. ~10 min.

Lesson 28 — AI & Data: Strike a Pose (train a custom classifier)

Objective (I can...): I can create and train my own classifier by giving it example photos. **CS concept:** Training a custom classifier (examples become training data) · **Builds on:** Lesson 27 (using classifiers). *Privacy: photos are captured on-device only, used just for training, and never saved or uploaded.*

AI Connection (2 min): Now YOU are the teacher — you show the AI examples so it learns, the payoff of every "AI learns by repeating examples" note this year. *First-principles question:* "How many examples of each pose do you think it needs to tell them apart?"

Materials: LEGO CS & AI kit + device per pair with camera, open floor space to pose. **New build for the train-your-own sub-unit** — build the Strike a Pose reactor model now and keep it assembled through Lesson 29.

60-minute flow

Warm-up (8): "Teach a friend" — one student describes a pose in words; partner copies. More clear examples make learning easier — same for AI.

Concept + demo (10): Explain training: show the classifier several photos of each pose so it learns to tell them apart. Demo capturing a few examples for two poses. Restate: images stay on-device, not saved.

Build/Code (30): Pairs **build the pose-reactor model**, create a custom classifier with 2 pose categories, photograph several examples of each themselves, then program the model to react to each recognized pose. **Examples are data:** on their plan sheet, pairs keep a simple tally of examples-per-category (reusing the counting/Variables skill) and ask "are my categories balanced?" *Swap at 15 min so both capture examples and code.*

Challenge (7): Add a third pose category; test whether the model still tells them apart.

Share + pack-up (5): Pairs demo their trained classifier recognizing their poses. **Leave the model built.**

Pair roles: Driver captures/labels example photos, Navigator codes the reactions. Swap at 15 min. *(Trio rotates pose/capture/code.)*

Differentiation: *Support* — start with two very different poses. *Extension / fast-finisher* — use subtle poses and add more examples. *Reteach* — capture examples together and watch the label appear correctly before coding.

Exit check: Pair trains a custom classifier that recognizes their own poses and triggers actions. **At-home (optional):** Practice 3 clear poses; think about how you'd describe each. ~10 min.

Lesson 29 — AI & Data: Copy That (improve a custom classifier)

Objective (I can...): I can improve a classifier's accuracy by training it with better examples. **CS concept:** Training quality and accuracy (more/varied examples improve results) · **Builds on:** Lesson 28 (training a classifier). *Privacy: on-device camera, no logins, images never saved.*

AI Connection (2 min): This is why we said "more tries help" back in the loops unit — better, more varied examples make AI more accurate. *First-principles question:* "What kind of example would fix a classifier that keeps guessing wrong?"

Materials: LEGO CS & AI kit + device per pair with camera. **Model reuse:** pose-reactor model from Lesson 28 — code and retrain only.

60-minute flow

Warm-up (8): "Tricky twins" — coach shows two similar gestures; ask how to tell them apart reliably. Connect to clearer, more varied examples.

Concept + demo (10): Demo a classifier that guesses wrong, then add more and better example photos; show accuracy improve.

Build/Code (30): Pairs train the classifier to reliably tell categories apart — test accuracy, add examples until it's dependable, then link recognitions to model actions. **Keep the data tally going** (examples-per-category) and re-check "are my categories balanced?" *Swap at 15 min.*

Challenge (7): Introduce a tricky look-alike category; train until the model handles it.

Share + discussion (5): Pairs report how accuracy changed, then discuss **why** the classifier still guesses wrong sometimes: did the training examples "represent everyone/everything"? (E.g., trained in only one kind of lighting, or on only one person, or with lopsided category counts.) Name one better example that would help. Model stays built.

Pair roles: Driver captures new examples, Navigator tracks and reports accuracy. Swap at 15 min. (*Trio: third runs the accuracy tests.*)

Differentiation: *Support* — focus on improving one category. *Extension / fast-finisher* — balance three categories for high accuracy. *Reteach* — add examples one at a time and re-test so the improvement is visible each time.

Exit check: Pair shows measurable improvement after adding/varying training examples, and can name one reason an AI might make a mistake and one way better examples would help. **At-home (optional):** Explain to a family member why AI needs many examples to learn well. ~10 min.

Lesson 30 — AI & Data: Story Studio (Design Challenge)

Objective (I can...): I can design an interactive story that uses AI to respond to the audience. **CS concept:** Applying trained classifiers with events, conditionals, loops, and variables · **Builds on:** Lessons 26–29 and the whole course. *Privacy: on-device camera, no logins, images never saved. Also serves as flex/catch-up — use today to finish and polish anything outstanding from the AI unit.*

AI Connection (2 min): Today your build senses, decides, counts, and responds to an AI YOU trained — every unit of the year working together. *First-principles question:* "Which idea from this year did your story rely on most?"

Materials: LEGO CS & AI kit + device per pair with camera, plan sheets, story props/cards. **Model already built** — reuse the pose-reactor (L28) or Monster model (L26), or a small reconfig; the 30 min is plan + code + test, not build-from-scratch.

60-minute flow

Warm-up (8): Story spark — pairs pitch a one-sentence story where the audience's pose or object changes what happens next.

Concept + demo (10): Challenge: build an interactive story where an AI recognition (pose or object) drives the model and the story. Coach models a short plan connecting a recognition to a story beat. Restate privacy points.

Build/Code (30): Pairs plan and code an interactive story using a classifier (pre-trained or their own) plus at least one conditional and, ideally, a variable or loop, on the already-built model. *Swap at 15 min.*

Challenge (7): Add a branching path so a different recognition leads to a different ending.

Share + pack-up (5): Story film festival — each pair performs their interactive story while the coach records a short video clip for the parent note. **Photograph each build.**

Pair roles: Co-plan; Driver codes / Navigator directs the story beats and cues recognitions; swap at 15 min. (*Trio rotates plan/code/perform.*)

Differentiation: *Support* — one recognition, one story beat, partial plan. *Extension / fast-finisher* — multiple recognitions with branching endings and a score. *Reteach* — reuse their Lesson 28–29 trained classifier and add just one story reaction.

Exit check: Pair delivers a working interactive story that uses AI recognition to change what happens, with a matching plan. **Progress layer:** Send home the **Unit 6 (course finale) parent note** ("your child can train an AI and build a machine that senses, decides, counts, and responds") with the **build photo** — the year's capstone.

At-home (optional): Storyboard a 3-scene interactive story; note where the audience chooses the path. ~10 min.

Standards Crosswalk

Aligned to the **CSTA K-12 CS Standards** — primarily **Algorithms & Programming (1B-AP)**, with connections to **Data & Analysis (1B-DA)** and **Impacts of Computing (1B-IC)**. This is a targeted map, not a claim of full coverage. *Maps to the Virginia CS Standards of Learning, grades 3–5 (confirm exact SOL numbers).*

Unit (Lessons)	Skills taught	CSTA 1B codes targeted
1 • Basics (1–5)	Sequencing, decomposition, pseudocode/algorithm planning, abstraction (plain-words plan)	1B-AP-11 (decompose problems into smaller parts); 1B-AP-10 (create programs with sequences); 1B-AP-08 (compare/refine multiple solutions — surfaced at design challenges)
2 • Events (6–10)	Event triggers; buttons, taps, and sensor inputs; multiple event handlers	1B-AP-10 (events/control structures); 1B-AP-11 (decomposition)
3 • Loops (11–15)	Repeat loops, forever/stop, nested loops (<i>stretch</i>), abstraction (a loop hides copied blocks), debugging	1B-AP-13 (use loops/iteration); 1B-AP-15 (test and debug); 1B-AP-10
4 • Conditionals (16–20)	If/then, true/false, if/then/else, sensor-threshold decisions; comparing two working solutions	1B-AP-10 (conditionals/control); 1B-AP-15 (debug); 1B-AP-08 (compare/refine solutions)
5 • Variables (21–25)	Create/store a variable, increment/counting, variables in decisions, multiple variables	1B-AP-09 (create programs that use variables); 1B-AP-10
6 • AI & Data (26–30)	Use a pre-trained classifier; train a custom classifier; examples as balanced data; accuracy, error, and fairness	1B-DA-06 (data used to make predictions / examples as training data); 1B-IC-18 (impacts, fairness, and who is helped or harmed); 1B-AP-10
Pair programming (every lesson)	Driver/Navigator roles, swapping, collaboration	1B-AP-16 (take on varying roles when collaborating)

Cross-cutting: debugging mini-tasks (L12, L14, L17, L19) reinforce **1B-AP-15**; the "two ways to solve it" and design-challenge share-outs reinforce **1B-AP-08**; the AI-impacts prompts (L8, L13, L18, L25) and L29 fairness discussion reinforce **1B-IC-18**.